

l: ta9ra menou
e: tfassa5 ili fih w tassen3ou men jdidd w tekseb fih melawel
a: ajouter fil deriner ligne fil fichier

“c:\\ doc.txt” : nom fichier chaîne de caractère : nom physique
nom logique: f

1) Soit F un fichier d'entiers, écrire l'algorithme qui permet de générer un fichier G contenant les carrés des éléments de F

algorithme fichier puissance

const:

NF: chaîne de caractère = "c:\\fichier source.txt"

NG: chaîne de caractère = "c:\\fichier puissance.txt"

var:

f,g: fichier text

v: entier

début

```
    assigner(NF,f);
    assigner(NG,g);
    ouvrir(f,l);
    ouvrir(g,e);
    tantque EOF(f) = faux faire
        lire(f,v);
        ecrire(g,v*v);
    fin tanque
    fermer (f);
    fermer (g);
```

fin

2) écrire un algo qui permet d'afficher un contenu d'un fichier text saisi par utilisateur
algorithme afficher

var:

NF: chaîne de caractère

f: fichier text

v: chaîne

début

```
    ecrire('donner nom de fichier');
```

```

lire(NF)
assigner(NF,f);
ouvrir(f,l);
tantque (non( EOF(f) ) faire
    lire(f,v);
    écrire(v);
fin tantque
fermer (f);
fin

```

1) écrire un algorithme qui à partir de fichier des caractères saisis par utilisateur d'afficher le dernier caractère :

algorithme affiche dernier caractère

var:

NF:chaîne de caractère

f;fichier de caractère

v:caractère

début

écrire ('donner nom de fichier');

lire(NF);

assigner(NF,f);

ouvrir(f,l);

tantque non(EOF(f)) faire

lire(f,v);

fin tantque

écrire('le dernier fichier ', v);

fermer(f);

fin

2) en vue fournir à partir de mem fichier de caractère un tableau contenant tous les caractère de fichier :

algorithme

type: **tab** : tableau [1..100] caractère

var:

NF:chaîne de caractère

f;fichier caractère

v:caractère

t:tab

début

 ecrire ('donner nom de fichier');

 lire(NF);

 assigner(NF,f);

 ouvrir(f,l);

$i \leftarrow 1$;

 tantque non(EOF(f)) faire

 lire(f,v);

$t[i] \leftarrow v$;

$i \leftarrow i+1$;

 fin tantque

 si i=0 alors

 ecrire('fichier vide');

 sinon

 ecrire(fichier est prêt');

 fermer(f);

fin

3) soit un fichier d'entier donner par l'utilisateur en veut fournir deux fichier à partir de premier contenant respectivement les entier paire et impaire

algorithme affiche dernier caractère

var:

NF:chaîne de caractère

paire:chaîne de caractère

impaire:chaîne de caractère

f;fichier de entier

p;fichier de entier

im:fichier entier

v:entier

début

 écrire ('donner nom de fichier ');

 lire(NF);

 assigner(NF,f);

 écrire ('donner nom de fichier ');

 lire(paire);

 assigner(paire,p);

 écrire ('donner nom de fichier ');

 lire(impaire);

 assigner(impaire,im);

 ouvrir(f,l);

 ouvrir(p,e);

 ouvrir(im,e);

 tantque non(EOF(f)) faire

 lire(f,v);

 si $v \bmod 2 = 0$ alors

 écrire (p,v)

 sinon

 écrire(im,v)

 fin si

 fin tantque

 fermer(p);

 fermer(im);

 fermer(f);

 ou *fermer(p,im,f);

fin

4) écrire un algorithme qui à partir de fichier étudiant fournie à un fichier contenant les nom des étudiant ayant une moyenne = 0 , chaque étudiant et écrire par son nom et son prenom sa moyenne

algorithme affiche dernier caractère

var:

structure : eleve =>nom ; chaine de caractère

=>prenom : chaine de caractère

=>moyenne : réel

NF:chaîne caractère

nz:chaîne caractère

f;fichier de eleve

el ;fichier de chaîne de caractère

v:entier

début

 écrire ('donner nom de fichier ');

 lire(NF);

 assigner(NF,f);

 écrire ('donner nom de fichier ');

 lire(nz);

 assigner(nz,el);

 ouvrir(f,l)

 ouvrir(el,e);

 tantque non.eof(f) faire

 lire(f,v);

 si v.moyenne=0 alors

 écrire(el,v.nom);

 fin si

 fin tantque

 fermer(el,f);

fin

5) ecrire un algorithme en veut fournir deux fichiers contenant le nom des etudiants respectivement ayan moyenne ≥ 10 et un moyenne <10 :

algorithme affiche dernier caractère

var:

structure : eleve =>nom ; chaine de caractère

=>prenom : chaine de caractère

=>moyenne : réel

NF:chaîne caractère

nad:chaîne caractère

ncr :chaîne caractère

f;fichier de eleve

ad ;fichier de chaîne de caractère

cr::fichier de chaîne de caractère

v:eleve

début

ecrire ('donner nom de fichier ');

lire(NF);

assigner(NF,f);

 ecrire ('donner nom de fichier ');

lire(nad);

assigner(nad,ad);

 ecrire ('donner nom de fichier ');

lire(ncr);

assigner(ncr,cr);

 ouvrir(f,l)

ouvrir(ad,e);

ouvrir(cr,e);

 tantque non eof(f) faire

lire(f,v);

si v.moyenne $\geq$ 10 **alors**

ecrire(ad,v.nom);

sinon

ecrire(cr,v.nom);

fin si

fin tantque

fermer(ad,cr,f);

fin

algorithme rigoureux :rapide ?!

gestion dynamique de la mémoire != gestion statique de la mémoire

pointeur:

algorithme gestion_liste

type:

cellule structure =>info:entier

=>suiv: pointeur sur cellule

var

l:pointeur sur cellule

q,p:pointeur sur cellule

début

l←allouer(cellule); ta3tini adresse ta3 lcase

ecrire('saisir info');

lire(l→info):

l→suiv←null

p← allouer(cellue);

ecrire("saisir info")

lire(p→info)

(p→suiv) ←null

#relation

l→suiv←p

q← allouer(cellule)

ecrire('donner info')

lire(q→info);

q→suiv←null

p→suiv←q

ecrire un algorithme qui permet de construire la première cellule d'une list d'entier

algorithme

```
type ;
    case structure >info:entier
                > suiv:pointeur sur case
    -----
    list structure >tête;pointeur
                >queue:pointeur sur case
                >nc:entier (nombre de cellule)

var;
    l:list
    p:pointeur sur case
début
    p←allouer(case);
    ecrire("saisir info");
    lire(p→info);
    p→suiv←null
    l.tete←p
    l.nc←1
    l.queue←p
fin ‘
```

ecrire un algorithme qui permet d'ajouter une case a la fin de a liste :

```
p←allouer(case)
ecrire("saisir info");
lire(p→info);
p→suiv←null
(l.queue →suiv)←p
l.nc←l.nc+1
l.queue←p;
```


ecire un algorithme qui permet d'ajouter 3 case a la de liste

```
var i;entier
début
    pour ide 1 a 3 faire
        p←allouer(case)
        ecrire ("saisir info ");
        lire(p→info);
        (l.queue→suiv) ←p
        l.queue ←p
        l.nc←l.nc+1
    fin pour
    l.queue →suiv ← null;
```

ecire un algorithme qui permet d'insérer une case au d"but de la liste

```
p← allouer(case)
lire(p→info)
p→suiv ← l.tete
l.tete ←p
l.nc←l.nc+1
```

ecrire un algorithme qui permet d'ajouter une case dans une position donnée par utilisateur (liste d'entier);

algorithme insert-pos

```
type ;
    case structure >info:entier
                > suiv:pointeur sur case
    -----
    list structure >tête;pointeur
                >queue:pointeur sur case
                >nc:entier (nombre de cellule)
var;
    l:list
    p,q:pointeur sur case
    pos:entier
début
    repeter
        écrire('donner position');
        lire (pos);
        jusqu'à pos>1 et pos <l.nc
        p← alouer(case);
        ecrire('saisir info')
        lire(p→info);
        q←l.tête
        pour i de 1 a pos-1 faire
            q←(q→siuv)
        fin pour
        (p→suiv)←(q→suiv)
        (q→suiv)←p
        l.nc←l.nc+1
fin
```

algorithme keemel fih les procédure kol

algorithme

```
type ;  
case structure >info:entier  
           > suiv:pointeur sur case
```

```
list structure >tête;pointeur  
              >queue:pointeur sur case  
              >nc:entier (nombre de cellule)
```

```
var;  
l:list
```

```
pos:entier  
procédure insert_pos(@l:list,pos:entier)  
var :
```

```
    p,q:pointeur sur case  
    i:entier
```

```
début
```

```
p← alouer(case);
```

```
ecrire('saisir info')
```

```
lire(p→info);
```

```
q←l.tête
```

```
pour i de 1 a pos-1 faire
```

```
    q←(q→siuv)
```

```
fin pour
```

```
(p→suiv)←(q→suiv)
```

```
(q→suiv)←p
```

```
l.nc←l.nc+1
```

```
fin
```

ecrire un algorithme pour supprimer la tête de la liste:

```
procédure sup_tête(@l:liste,@val:entier);
```

```
var
```

```
    p:pointeur sur case
```

```
début
```

```
    p←l.tete
```

```
    val ← (p →info)
```

```
    l.tête ←-(l.tete →suiv)
```

```
    liberer(p);
```

```
    l.nc ←( l.nc -1)
```

```
fin
```

ecrire un algorithme pour supprimer la queue de la liste:

```
procédure sup_queue(@l:liste,@val:entier);
```

```
var
```

```
    p:pointeur sur case
```

```
début
```

```
    p←l.queue
```

```
    val ← (p →info)
```

```
    q← l.tete
```

```
    pour ide 1 a l.nc -1 faire
```

```
        q←(q →> suiv)
```

```
    fin
```

```
    l.queur ← q
```

```
    (l.queue → suiv) ←null
```

```
    libérer(p);
```

```
    l.nc ←( l.nc -1)
```

```
fin
```

ecrire un algorithme pour supprimer position donnée par l'utilisateur

procédure sup_queue(@l:liste,@val:entier , pos ;entier);

var

i:entier

p1:pointeur sur case

p2:pointeur sur case

début

p1 ← l.tete

pour i de 1 a pos-1 faire

p1 ← (p1 → suiv)

fin

p2 ← (p1 → suiv) ;

val ← p1 → val;

(p1 → suiv) ← (p2 → suiv)

libérer(p2);

l.nc ← (l.nc -1)

fin

ecrire un algorithme qui permet de fournir une liste identique a cell d'entier
algorithme

type ;

case structure >info:entier
 > suiv:pointeur sur case

list structure >tête;pointeur
 >queue:pointeur sur case
 >nc:entier (nombre de cellule)

var;

l1:list

l2:list

procédure copier(l1:list,@l2:list)

var :

p,q,z:pointeur sur case

i:entier

début

p← allouer(case)

(p→info) ← (l1.tête → info)

l2.tête ←p

l2.queue←p

[l2.nc](#)←1

z←(l1.tête)

pour i de 2 a [l1.nc](#) faire

 q← allouer(case)

 (p→suiv) ←q

 z←(z→suiv)

 (q→info)←(z→info)

 l2.queue← q

[l2.nc](#)←l2.nc+1

fin pour

(l2.queue →suiv) ←null

fin

Procédure copier(l1: list, var l2: list)

var :

p, q, z: pointeur sur case

i: entier

début

p ← allouer(case) [cite: 85]

p→info ← l1.tête→info

l2.tête ← p

l2.queue ← p

l2.nc ← 1

z ← l1.tête→suiv // On se positionne sur la 2ème cellule de l1

// Boucle pour copier le reste de la liste

pour i de 2 a l1.nc faire

q ← allouer(case) [cite: 85]

q→info ← z→info

// On lie l'ancienne cellule (p) à la nouvelle (q)

p→suiv ← q

// On avance les pointeurs

p ← q // p devient la nouvelle cellule pour le prochain lien

z ← z→suiv // on avance dans l1

// Mise à jour des métadonnées de l2

l2.queue ← q

l2.nc ← l2.nc + 1

fin pour

// La dernière cellule doit pointer vers null [cite: 87, 89]

p→suiv ← null

fin si

fin

crée 1 ere case :

algorithme liste-double-chaîne

type case structure =>prec:poiteur
=>info:entier
=>suiv :pointeur sur case

liste structure =>tête :pointeur sur case
=>nc : entier
=>queue:pointeur sur case

procédure construction tête(@l:list,info:entier)

var

p:pointeur sur case

début

p← allouer(case)

(p→info)← info

(p→prec) ←null

(p→suiv)←null

l.tête←p

l.queue←p

[l.nc](#)←1

fin

procédure insert-fin(@l:list,A:entier)

var

p:pointeur sur case

début

p←allouer(case);

p→suiv←null;

p←prec←l.queue;

p→info←A

l.queue→suiv←p;

l.queue←p

l.nc←l.nc+1

fin

wa9teh nesta3emlou les listes fi blasset tableau = wa9teli bech tamel barcha des mises a jour

1) **ecrire un sous programme qui permet d'ajouter une case au début d une liste donnée**

procédure insert_début(@l:list , A:entier)

var :

p:pointeur sur case

début

p ← alouer(case);

p → info ← A

p → prec ← null

p → suiv ← l.tête

l.tête → prec ← p

l.tête ← p

l.nc ← l.nc +1

fin

2) **ecrire un sous programme qui permet ajouter un case a un position donner :**

procédure ajouter_case(@l:liste, pos:entier, val:entier)

var

p,q:pointeur sur case

début

p ← alouer(case)

p → info ← val

q ← l.tête

pour i de 1 a pos -1 faire

q ← (q → suiv)

fin pour

p → prec ← q

p → suiv ← (q → suiv)

((q → suiv) → prec) ← p

q → suiv ← p

l.nc ← l.nc +1

fin

3) écrire un sous programme qui affiche la liste inversée:

procédure afficher_liste_inversée(l:liste)

var

p:pointeur

val:entier

début

p←l.queue

pour i de 1 à l.nc faire

 écrire(p→info);

 p←(p→prec)

fin pour

fin

4) écrire un sous programme qui permet de fournir le nombre d'occurrence d'un caractère saisi par utilisateur dans une liste simple chaîne

fonction nb_occurrence(l:list , car:caractère):entier

var :

p:pointeur sur case

nb:entier

début

p←l.tête

nb←0

pour i de 1 à l.nc faire

 si p→info =car alors

 nb← nb+1

 fin si

 p←(p→suiv)

fin pour

return nb

fin

5) copier une liste simple chainer

procédure copier(l1:liste , @l2:liste)

var

p,q:pointeur sur case

i:entier

début

p←allouer (case)

q ←- (l1.tete)

l2.tête←p

l2.queue←p

l2.nc←5) copier une liste simple chainer

procédure copier(l1:liste , @l2:liste)

var

p,q:pointeur sur case

i:entier

début

p←allouer (case)

q ←- (l1.tete)

l2.tête←p

l2.queue←p

l2.nc←1

p→info←q→info

pour ide 2 a [l1.nc](#) faire

p←allouer(case);

q←q→suiv;

0

p→info←q→info

l2.queue→suiv←p

l2.queue←p

l2.nc←l2.nc+1

fin pour

l2.queue→suiv←null

fin

- 1) écrire un sous programme qui permet de supprimer la valeur minimale d'une liste entière simplement chaînée

procédure `supp_min(@l:liste)`

début

```
    admin ← l.tête
    q ← admin → suiv
    j ← 0
    pour i de 1 à l.nc faire

        si admin → info > q → info alors
            j ← i
            admin ← q
        fin si
        q ← q → suiv
    fin pour

    si j ≠ 1 et j ≠ l.nc alors
        q ← l.tête
        pour i de 1 à j-1 faire
            q ← (q → suiv)
        fin
        q → suiv ← (admin → suiv) ;
        l.nc ← (l.nc - 1)
        libérer(admin);
    sinon si j = 1 alors
        l.tête ← l.tête → suiv
        libérer(admin)
        l.nc ← l.nc - 1
    sinon
        q ← l.tête
        pour i de 1 à l.nc-1 faire
            q ← q → suiv
        fin pour
        l.queue ← q
        l.queue → suiv ← null
        l.nc ← l.nc - 1
        libérer(admin)
    fin
```

2) écrire un sous programme qui permet de vérifier si deux liste entier sont identique ;

fonction comparaison (l1:liste , l2:liste) : booléen

début

test ← vrai

si [l1.nc](#) = [l2.nc](#) alors

q ← l1.tête

p ← l2.tête

i ← 1

repete

si p → info ≠ q → info alors

test ← faux

sinon

q ← q → suiv

p ← p → suiv

i ← i + 1

fin si

jusqu'à test = faux ou i > l1.nc

sinon

test ← faux

fin si

si test = vrai alors

ecrire("les deux liste sont identiques");

sinon

ecrire("les deux liste ne sont pas identiques");

fin si

retourner test

fin

3) écrire un sous programme qui permet d'insérer une valeur donnée dans une liste triée

procédure insertion(@l:liste, val :entier)

début

pos ← 1

test ← faux

p ← l.tete

repete

si p → info > val alors

test ← vrai

sinon

pos ← pos + 1

p ← p → suiv

fin si

jusqu'à test = vrai ou pos > [l.nc](#)

q ← l.tete

pour i de 1 à pos - 1 faire

q ← (q → suiv)

fin pour

(p → suiv) ← q → suiv

(q → suiv) ← p

p ← alouer(case)

p → info ← val

[l.nc](#) ← l.nc + 1

fin

